



INDIA.RUNS

Build what next India runs on

Team Name : [Hack_IT_On](#)

Team Leader Name : [Pratham Modi](#)

Problem Statement : [Intelligent Candidate Discovery & Ranking - rank 100,000 candidates against a Senior AI Engineer JD and return a trustworthy, explainable top-100 shortlist.](#)

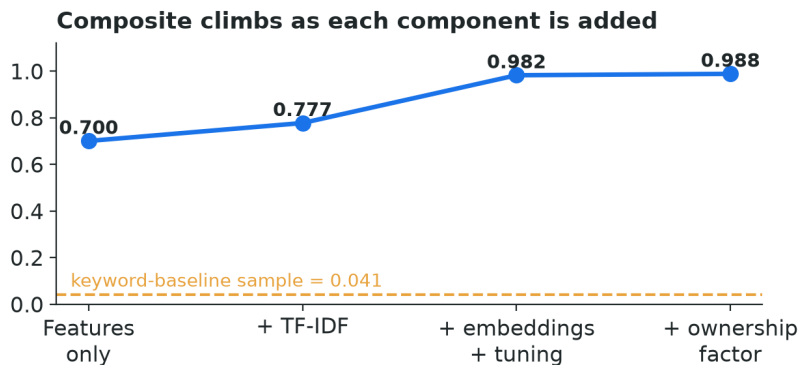
Solution Overview

- **What is our solution?**

- A recruiter-style ranking funnel: parse the JD into a structured spec, gate out irrelevant and internally-impossible profiles, then score survivors with a multiplicative blend of factors.
- Hybrid relevance = corroborated career-evidence + TF-IDF + sentence embeddings. Runs 100,000 to 100 in ~14s, CPU-only, no network.

- **What makes it different**

- Scores what candidates DID (career history), not the self-reported skills list, so keyword stuffing fails.
- Multiplicative scoring: an unreachable or inconsistent candidate cannot be rescued by a strong skill match alone.
- Every ranked candidate carries an auto-generated, fact-checked justification.



JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD**

- Experience 5-9 yrs (ideal 6-8); Pune/Noida preferred, metros welcome, no visa sponsorship; notice ≤ 30 days.
- Must-haves: embeddings/retrieval, vector-search infra, ranking systems, ranking evaluation, strong Python. Nice-to-haves: LLM fine-tuning, learning-to-rank, HR-tech domain, scale.

Most important candidate signals

- Production evidence of retrieval/ranking work in the career history; experience-band fit; behavioral availability (reply rate, recency, notice, open-to-work).

Evaluating fit beyond keywords

- A listed skill counts only when corroborated by an assessment score or real usage duration; embeddings surface strong candidates who use zero buzzwords.

Ranking Methodology

- **Retrieve, score, rank**

- Funnel: 100,000 -> title gate (-68,821) -> consistency gate (-482) -> 17,391 scored -> top 100.

$\text{score} = \text{skill_fit} \times \text{experience}^w \times \text{location}^w \times \text{availability}^w \times \text{ownership}^w \times \text{penalties}^w$

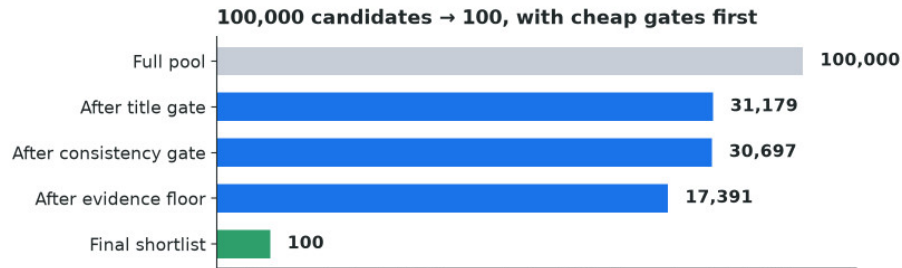
$\text{skill_fit} = 0.15 \times \text{evidence} + 0.41 \times \text{TF-IDF} + 0.33 \times \text{embedding}$

Models, algorithms, heuristics

- MiniLM (all-MiniLM-L6-v2) embeddings, scikit-learn TF-IDF, deterministic feature functions. No LLM at rank time.

Combining signals into one ranking

- Multiplicative product of normalized factors; exponents tuned by random search and minimax over six label hypotheses, against an offline benchmark on the official metric.



Explainability & Data Validation

- **How ranking decisions are explained**

- Each candidate's 1-2 sentence justification is assembled from the exact factors that set its score: real technologies found in their career text, real signal values, and honest concerns (relocation, notice period, gaps).

Preventing hallucination

- Reasoning cites only facts present in the profile. An automated audit re-checks every number, employer, date and technology on all 100 rows against the source JSON; all six official checks pass.

Handling suspicious or low-quality profiles

- A consistency gate discards internally-impossible profiles (claimed years > career span, expert skills never used, summary contradicting the experience field). Zero honeypots in our top-100.

Stage-4 reasoning audit — all 100 rows

✓ Specific facts	PASS	✓ No hallucination	PASS
✓ JD connection	PASS	✓ Variation	PASS
✓ Honest concerns	PASS	✓ Rank consistency	PASS

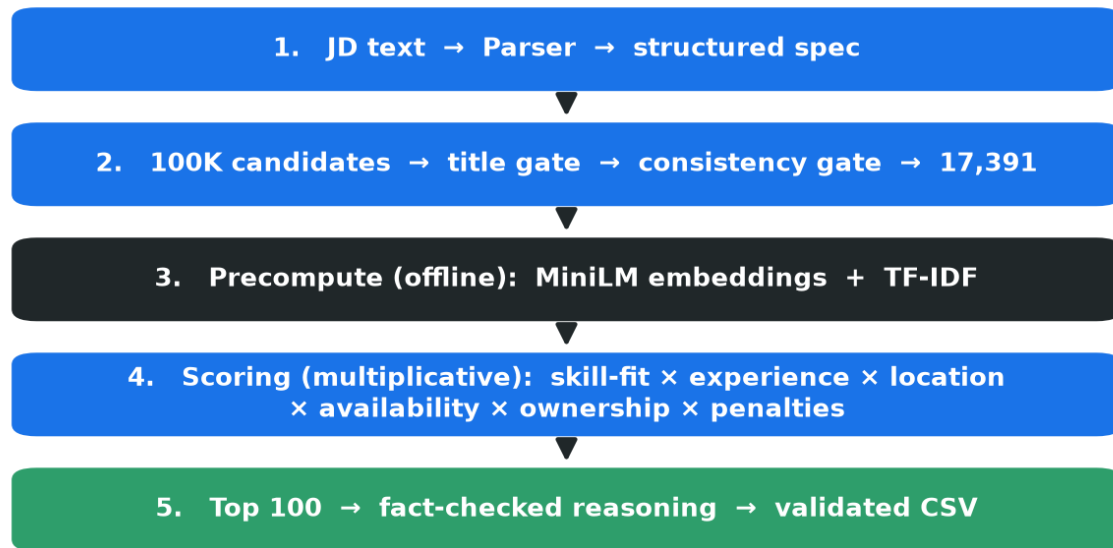
0 honeypots in top-100 · every claim verified against the profile JSON

End-to-End Workflow

- **JD input to ranked output**

1. Parse JD text into a structured spec (bands, location, competencies, disqualifiers).
 2. Pre-compute once, offline: MiniLM embeddings + TF-IDF over the 100K career texts.
 3. Stream candidates -> title gate -> consistency gate -> evidence floor.
 4. Score survivors with the multiplicative factor formula (tuned weights).
 5. Take the top 100; generate fact-checked reasoning per candidate.
 6. Emit ranked CSV; validate format and audit reasoning (one command: make judge).
- Ranking step: ~14s, CPU-only, no network calls.

System Architecture



Results & Performance

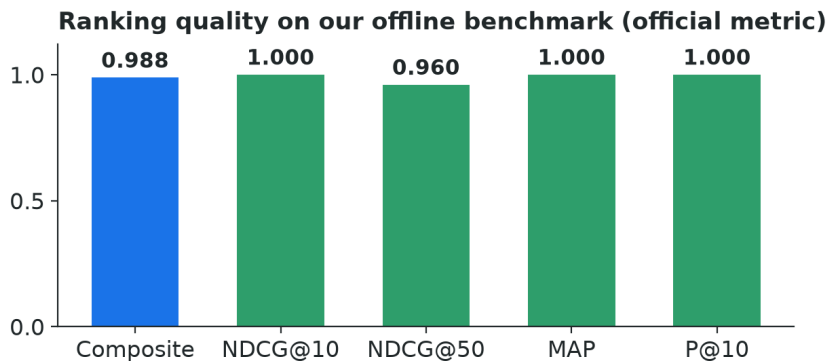
- Ranking quality (offline benchmark, official composite)**

- Composite 0.988 (worst-case 0.987 across six label hypotheses); NDCG@10 1.000, MAP 1.000, P@10 1.000, NDCG@50 0.960.
- 0 honeypots in top-100 (disqualification threshold is >10%). The organizers' sample ranking scores 0.041 on the same benchmark.
- +/-20% weight perturbation: top-100 stays 97.8% stable.

Meeting runtime and compute constraints

- Reproduced in Docker (python:3.11-slim, --network none, -m 16g): byte-identical CSV, ~50s wall-clock, <1GB RAM.
- Well inside the limits: 5 min / 16 GB / CPU-only / no network.

All scores are on our own benchmark built on the official metric; official scores are hidden until submissions close.



Technologies Used

- **Technologies and why**

- Python 3.11 - pipeline, gates, scoring.
- sentence-transformers (all-MiniLM-L6-v2) - CPU embeddings for semantic match; small and fast, saturates NDCG@10 at ~4x less compute than larger models.
- scikit-learn TF-IDF - rare-term lexical signal (FAISS, NDCG) that embeddings miss.
- NumPy - vectorized scoring over 17K survivors in seconds.
- Docker - reproducible, network-isolated evaluation environment.
- Streamlit - hosted sandbox demo.
- Chosen for CPU-only, offline, sub-5-minute operation; no LLM at rank time, by design and by rule.

Submission Assets

Submission assets

- GitHub repo: <https://github.com/pratham0039/candidate-ranking>
- Ranked output: submission.csv (top-100, passes the official validator).
- Precomputed artifacts: GitHub Release artifacts-v1 (embeddings, TF-IDF, JD embedding) + precompute.py to rebuild.
- Sandbox demo: <https://candidate-ranking-bhxswwstcvwappasices3cff.streamlit.app/>
- Reproduce: `python rank.py --candidates candidates.jsonl --jd data/job_description.md --out submission.csv` (or: make judge).
- Docs: README, experiments.md, docs/design_decisions.md, docs/how_it_works.html.



INDIA.RUNS

Build what next India runs on

THANK YOU